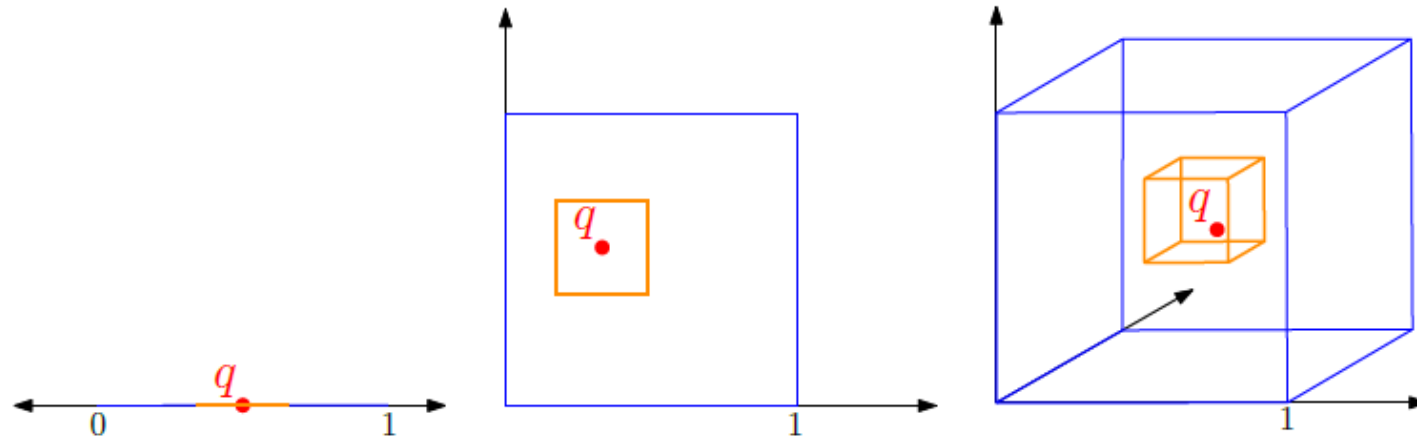


Lecture 15: Linear Dimensionality Reduction Methods

Instructor: Sergei V. Kalinin

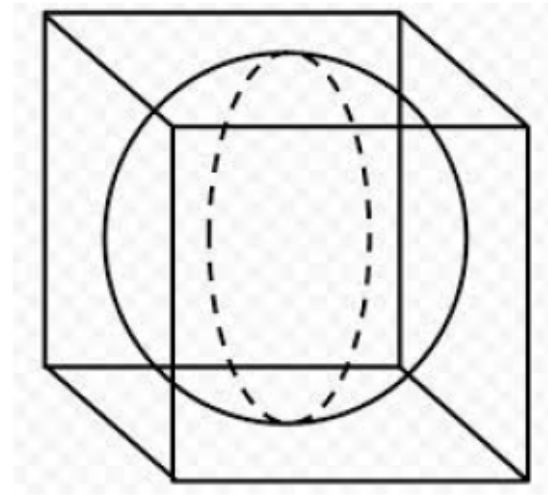
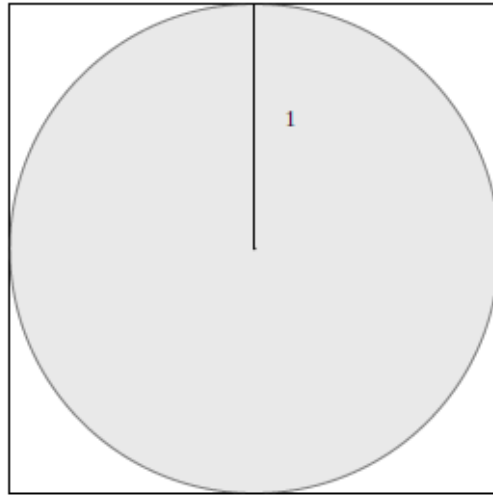
Reminder: dimensionality is a problem



Suppose we have 5000 points:

- In 1d we have to explore 0.001 on average to capture 5 NN
- In 2d, on average we must explore 0.031 units along both dimensions to get 5 nearest neighbors points (about 3% of the whole cube).
- In 3d, on average we must go 10% of the total (unit) length in each of the 3 dimensions
- In 4d, we must explore 17.7% of unit length
- In 10d, we must go 50.1% of unit length along each dimension

Reminder: dimensionality is a problem



dim m	volume of m -ball	volume of m -cube	ratio
2	π	2^2	~ 0.785
3	$4/3\pi$	2^3	~ 0.523
4	$\pi^2/2$	2^4	~ 0.308
6	$\pi^3/6$	2^6	~ 0.080
m	$\frac{\pi^{m/2}}{m/2!}$	2^m	$\rightarrow 0$

Reminder: dimensionality is a problem

However if a dataset exhibit this phenomenon that the issue has be overcome by getting a larger training set (exponential in m). One way to look at this is as follows.

To cover $[-1, 1]^m$ with $B_{m,1}$'s, the number of balls n must be

$$n \geq \frac{2^m}{V_m(1)} = \frac{2^m}{\pi^{m/2}/m/2!} = \frac{m/2! 2^m}{\pi^{m/2}} \stackrel{m \rightarrow \infty}{\sim} \sqrt{m\pi} \left(\frac{m2^{m/2}}{2\pi e} \right)^{m/2}$$

For $m = 16$ (a very small number) this n is substantially larger than 2^{58}

- In higher dimensions all the volume is in 'corners'
- Points in high dimensional spaces are isolated (empty surrounding)
- The probability that a randomly generated point is within r radius of q approaches 0 as dimensionality increases
- The probability of a close nearest neighbor in a data set is very small

High Dimensional Data – what should we do?

Examples of high D data:

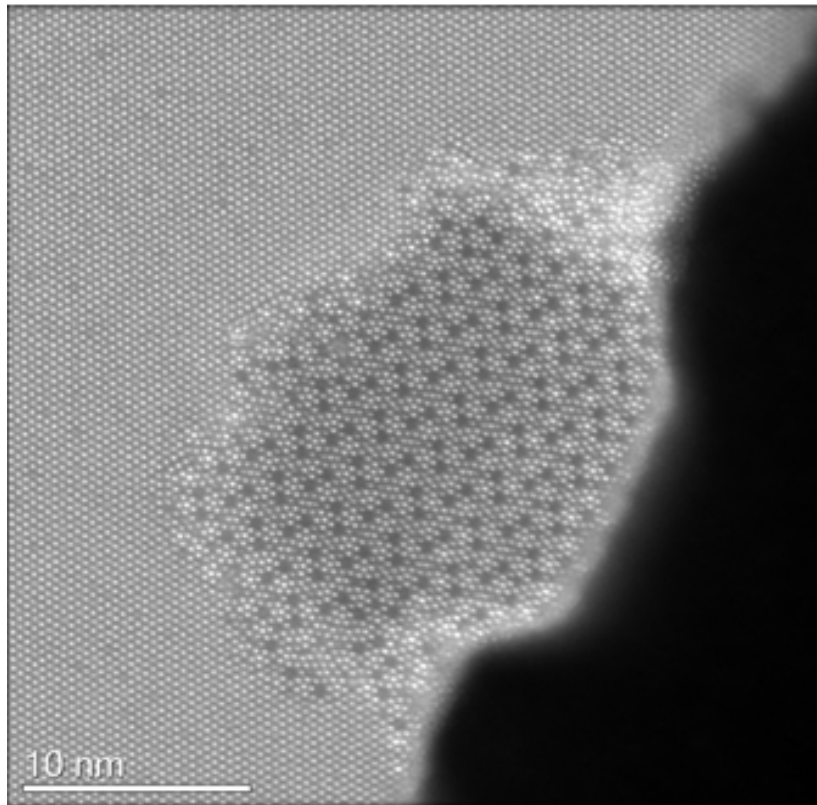
- Face recognition
- Image compression
- Gene expression analysis
- Spectroscopy
- 4D STEM
- X-Ray scattering

What do we want to accomplish?

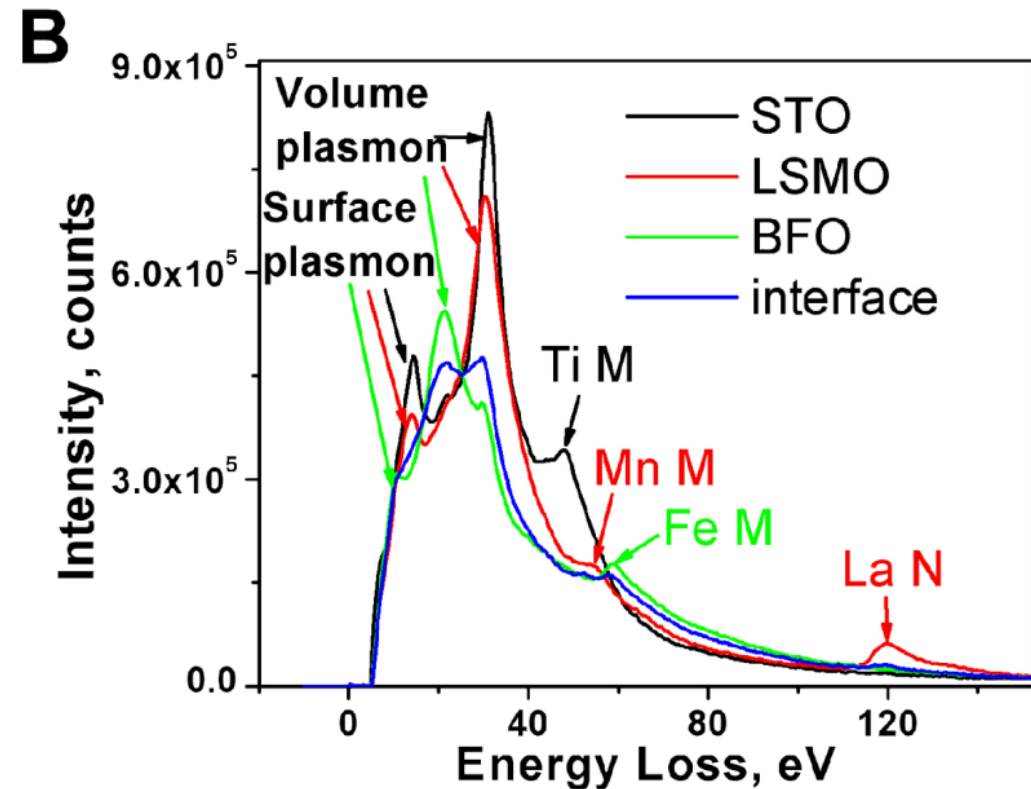
- Reduce number of dimensions in data
- Find patterns in high-dimensional data
- Visualize data of high dimensionality

High D data in materials science?

Mo-V-Ta complex oxide



Low-loss EELS spectra



Q. He et al, ACS Nano 9, 3470-3478

A.Y. Borisevich et al., Phys. Rev. Lett. **105**, 087204 (2010).

- How many dimensions are in this data?
- Are all these dimensions necessary?
- For given acquisition time, how would the noise and signal balance?
- How do we extract “useful” information?

High Dimensional Data is often redundant

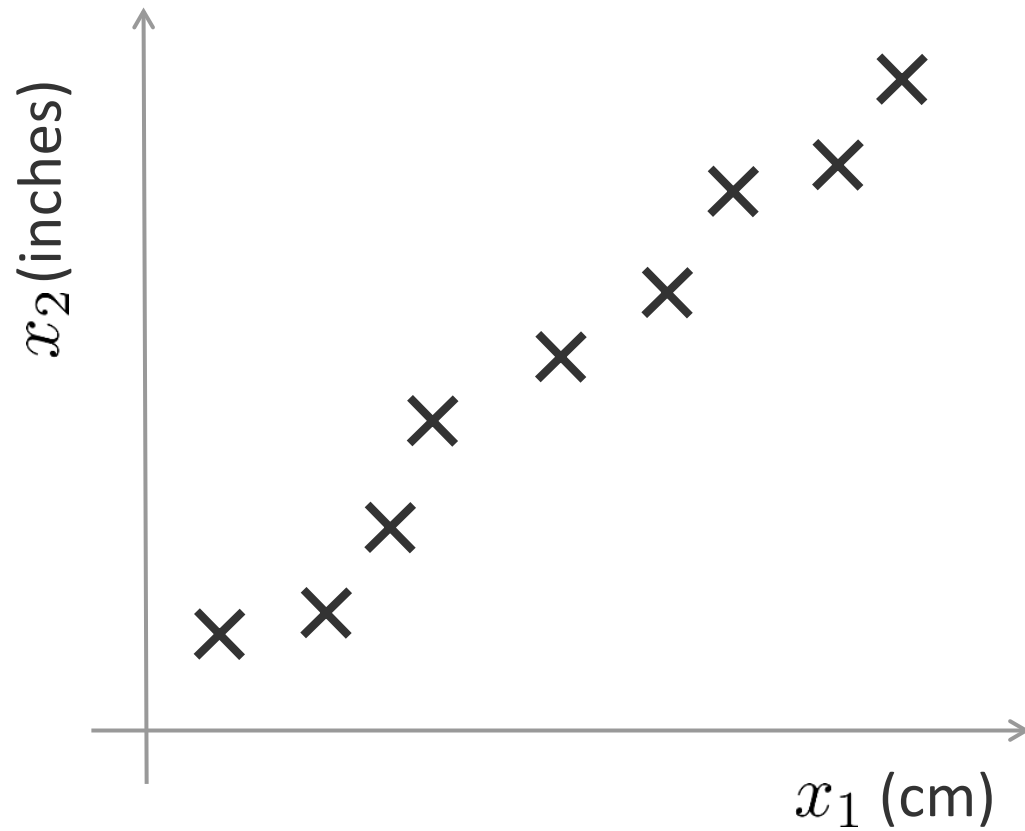
- We often need a method to simplify data on a large number of variables, and believe that there is some redundancy in those variables.
- Redundancy means that some of the variables are correlated with one another, possibly because they are measuring the same object or phenomenon.
- Because of redundancy, we believe that it should be possible to reduce the observed variables into a smaller number of artificial variables that will account for most of the variance in the observed variables.

Dimensionality Reduction Methods

- PCA (Principal Component Analysis):
 - Find projection that maximize the variance
- ICA (Independent Component Analysis):
 - Very similar to PCA except that it assumes non-Gaussian features
- Multidimensional Scaling:
 - Find projection that best preserves inter-point distances
- LDA (Linear Discriminant Analysis):
 - Maximizing the component axes for class-separation
- Bayesian Linear Unmixing
 - Linear unmixing, non-negative, sum to one
- ... constrained linear unmixing methods

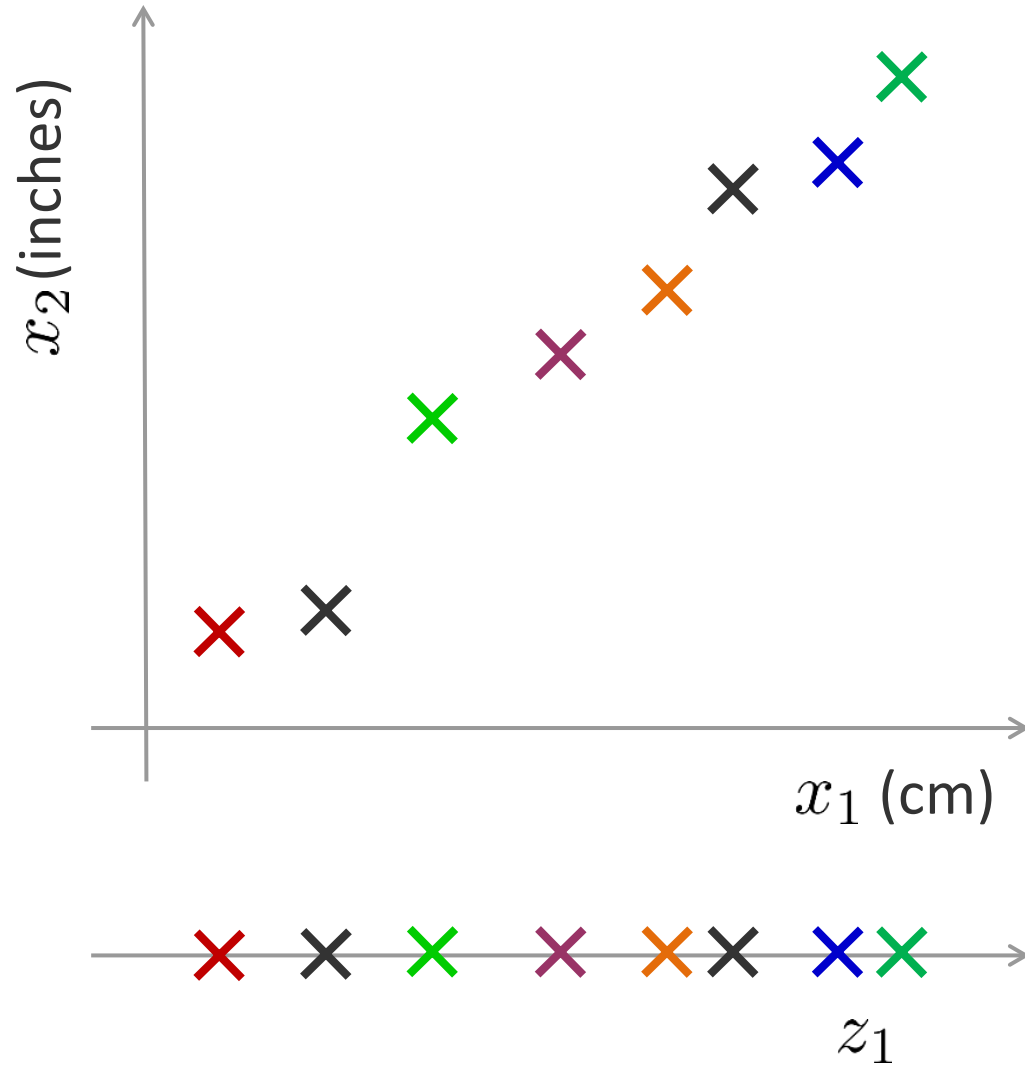
Manifold Hypothesis!

Simple Example



Reduce data from 2D to 1D

Simple Example



Reduce data from 2D to 1D

$$x^{(1)} \rightarrow z^{(1)}$$

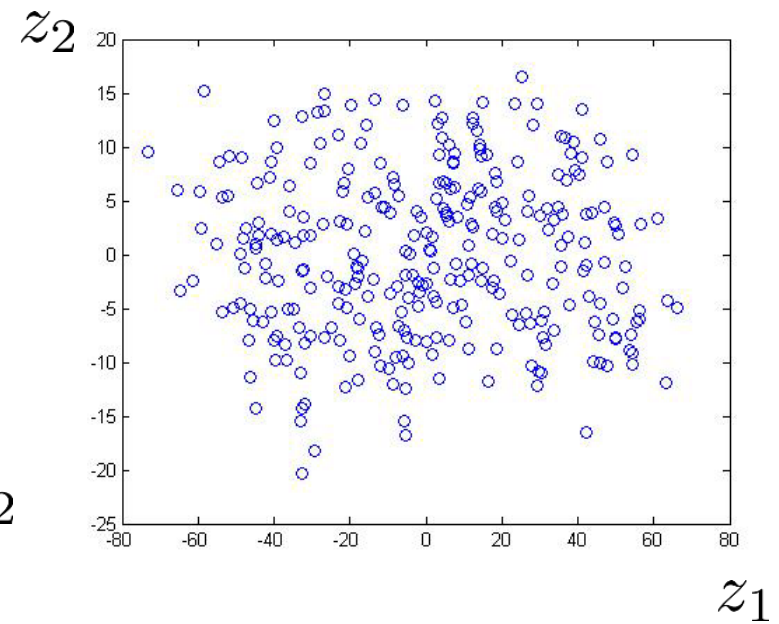
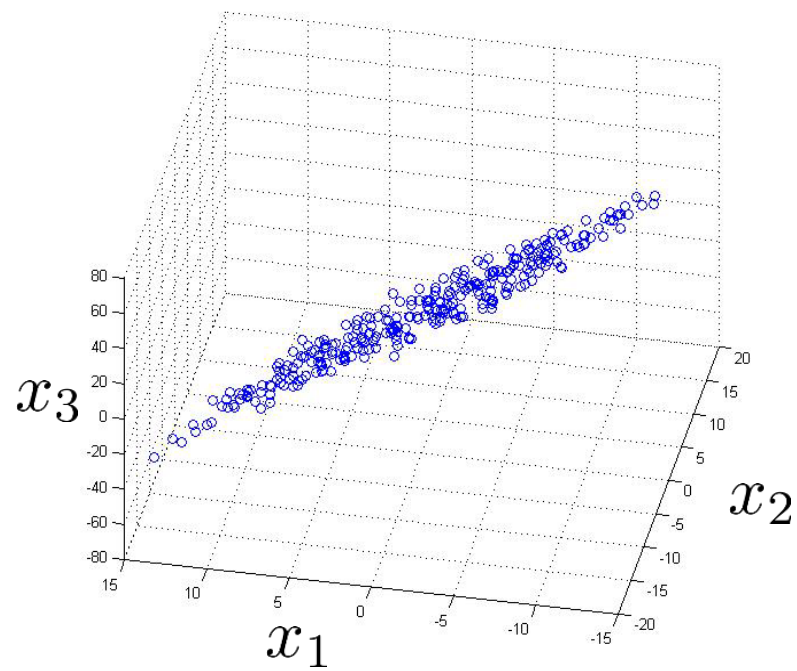
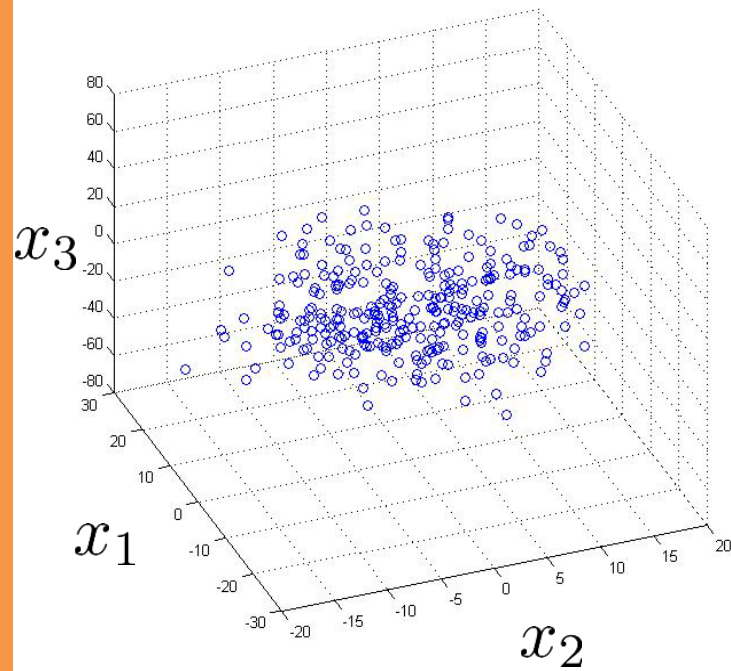
$$x^{(2)} \rightarrow z^{(2)}$$

$\vdots$

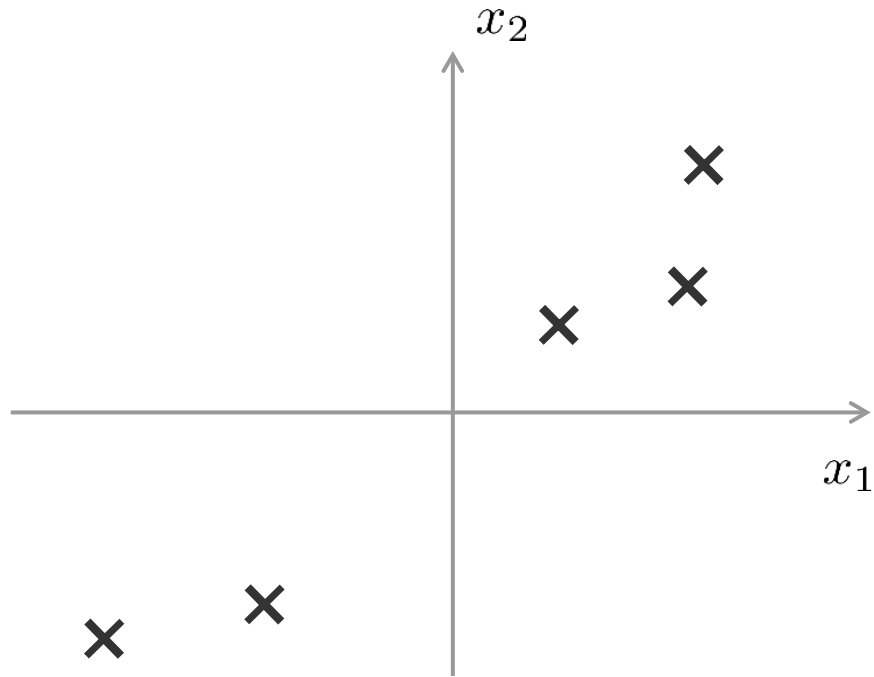
$$x^{(m)} \rightarrow z^{(m)}$$

Another Simple Example

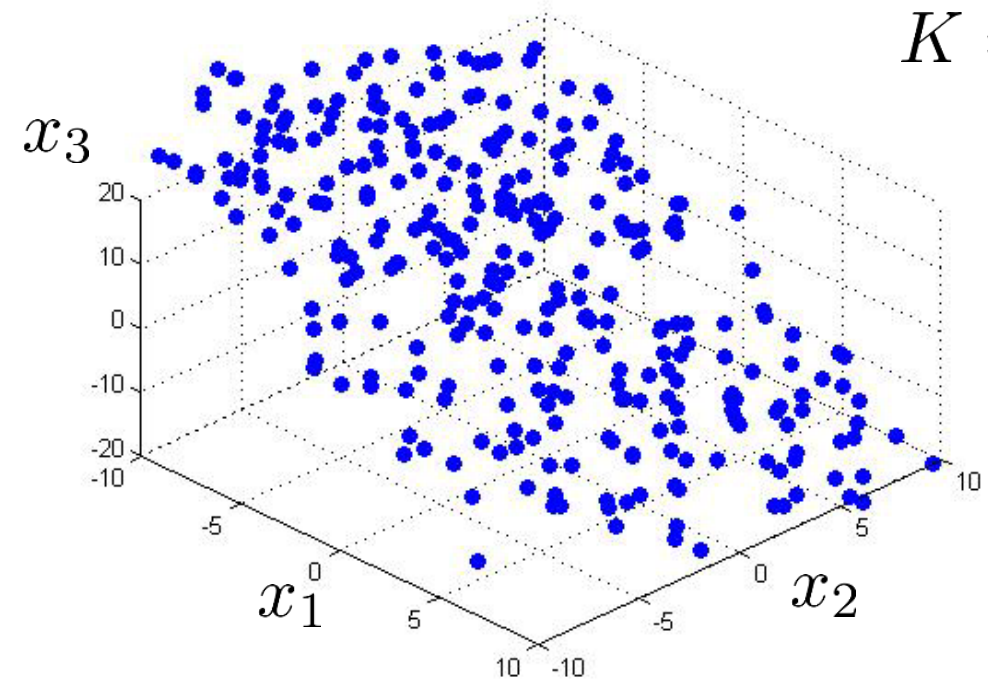
Reduce data from 3D to 2D



Generalize the problem



$$3D \rightarrow 2D$$
$$K = 2$$



Reduce from 2-dimension to 1-dimension: Find a direction (a vector $u^{(1)} \in \mathbb{R}^n$) onto which to project the data so as to minimize the projection error.

Reduce from n -dimension to k -dimension: Find k vectors $u^{(1)}, u^{(2)}, \dots, u^{(k)}$ onto which to project the data, so as to minimize the projection error.

Variance and covariance

1D: Variance=(Standard deviation)²

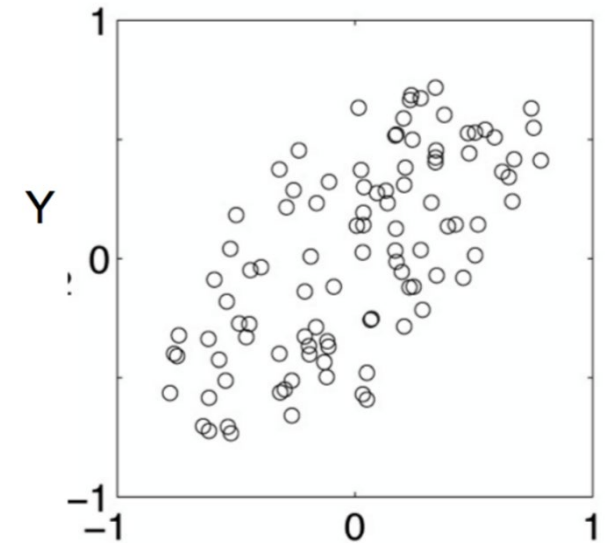
$$s^2 = \frac{\sum_{i=1}^n (X_i - \bar{X})^2}{(n-1)}$$

2D: Covariance: measures the correlation between X and Y

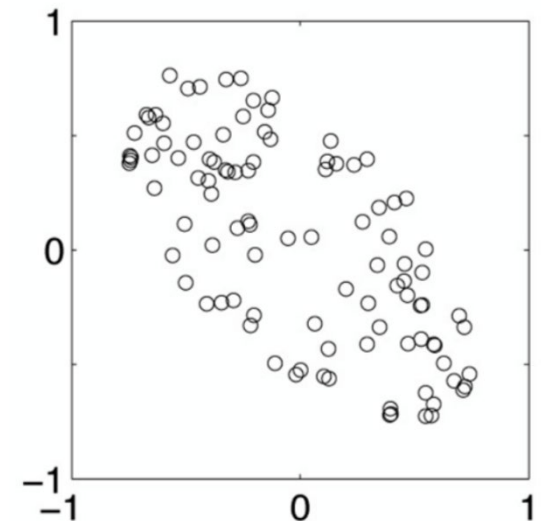
- Cov(X,Y)=0: independent
- Cov(X,Y)>0: move in the same direction
- Cov(X,Y)<0: move in opposite direction

$$\text{cov}(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{(n-1)}$$

positive covariance



negative covariance



Multidimensional data: covariance matrix

- Contains covariance values between all possible dimensions (=attributes):

$$C^{n \times n} = (c_{ij} \mid c_{ij} = \text{cov}(Dim_i, Dim_j))$$

- Example for three attributes (x,y,z):

$$C = \begin{pmatrix} \text{cov}(x, x) & \text{cov}(x, y) & \text{cov}(x, z) \\ \text{cov}(y, x) & \text{cov}(y, y) & \text{cov}(y, z) \\ \text{cov}(z, x) & \text{cov}(z, y) & \text{cov}(z, z) \end{pmatrix}$$

- Eigenvalues of covariance matrix contain information on the independent factors of variability
- Eigenvectors of covariance matrix provide the information on directions

Principal Component Analysis

- Center the data (subtract the mean $\boldsymbol{\mu} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$ from each data point)
- Compute the $D \times D$ covariance matrix $\mathbf{S}$ using the centered data matrix $\mathbf{X}$ as
$$\mathbf{S} = \frac{1}{N} \mathbf{X}^\top \mathbf{X} \quad (\text{Assuming } \mathbf{X} \text{ is arranged as } N \times D)$$
- Do an eigen decomposition of the covariance matrix $\mathbf{S}$ (many methods exist)
- Take top $K < D$ leading eigenvectors $\{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K\}$ with eigen values $\{\lambda_1, \lambda_2, \dots, \lambda_K\}$
- The K -dimensional projection/embedding of each input is $\mathbf{z}_n \approx \mathbf{W}_K^\top \mathbf{x}_n$
- Where $\mathbf{W}_K = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K]$ is projection matrix of size $D \times K$

Singular Value Decomposition

- If we just use the top $K < \min\{N, D\}$ singular values, we get a rank- K SVD

$$\mathbf{X} \approx \hat{\mathbf{X}} = \sum_{k=1}^K \lambda_k \mathbf{u}_k \mathbf{v}_k^T = \mathbf{U}_K \mathbf{\Lambda}_K \mathbf{V}_K^T$$

reconstruction error $\|\mathbf{X} - \hat{\mathbf{X}}\|$

- Fact: SVD gives the best rank- K approximation of a matrix
- PCA is done by doing SVD on the covariance matrix $\mathbf{S}$ (left and right singular vectors are the same and become eigenvectors, singular values become eigenvalues)

Principal Component Analysis

PCA: orthogonal transformation converting possibly correlated variables into linearly uncorrelated *principal components*

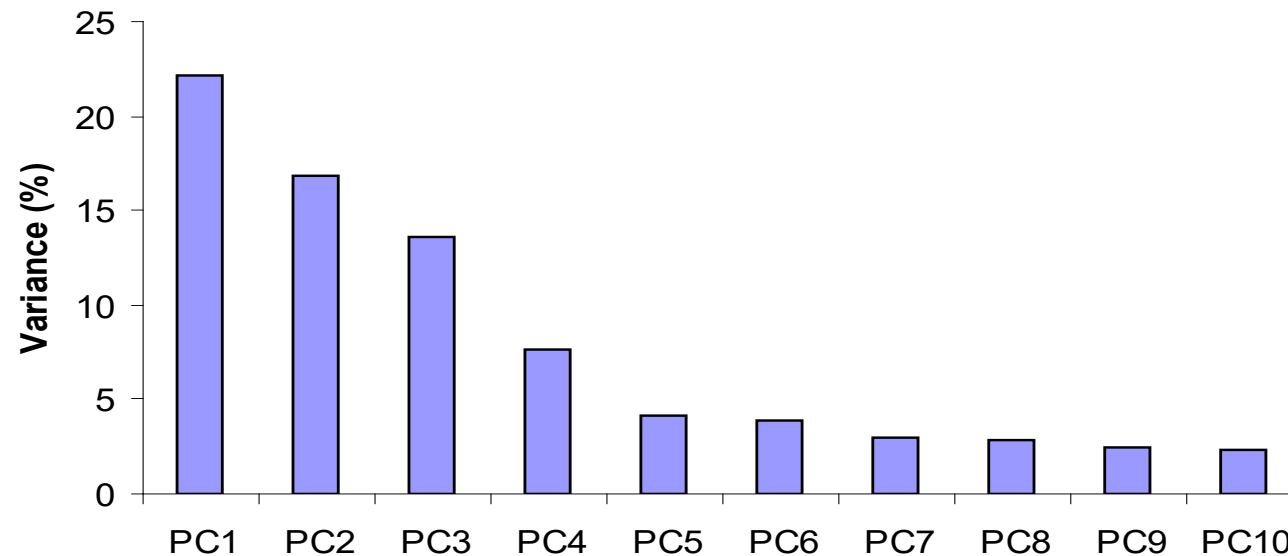
- PCA was invented by Karl Pearson in 1901, however the Singular Value Decomposition was independently derived some half a century earlier in Italy, Germany and France
- PCA transforms the data such that the greatest variance by any projection lies on the first coordinate
- Reveals internal structure of the data that best explains variance in the data set
- Since data often moves in clusters, PCA reveals those variables that drive the variance

Pearson, K. (1901). "On Lines and Planes of Closest Fit to Systems of Points in Space". *Philosophical Magazine Series 6* **2** (11): 559–572

PCA: Eigenvalues

Eigenvalues λ_j are used for calculation of [% of total variance] (V_j) for each component j :

$$V_j = 100 \cdot \frac{\lambda_j}{\sum_{x=1}^n \lambda_x} \qquad \sum_{x=1}^n \lambda_x = n$$



PCA: Components

- The first PC retains the greatest amount of variation in the sample
- The k^{th} PC retains the k^{th} greatest fraction of the variation in the sample
- The k^{th} largest eigenvalue of the correlation matrix C is the variance in the sample along the k^{th} PC
- The least-squares view: PCs are a series of linear least squares fits to a sample, each orthogonal to all previous ones

PCA Components and Loadings

- Technique useful for compression and classification of data
- Find new descriptors smaller than original variables
- Retain most of sample's information - correlation between original variables
- New descriptors are principal components (PCs)
- Loadings represent the “fraction” of PCs in initial data
- Uncorrelated, and ordered by fraction of total information retained in each PC

PCA in scikit-learn

sklearn.decomposition.PCA

```
class sklearn.decomposition.PCA(n_components=None, *, copy=True, whiten=False, svd_solver='auto', tol=0.0,
                                iterated_power='auto', n_oversamples=10, power_iteration_normalizer='auto', random_state=None)
```

[\[source\]](#)

Methods

<code>fit(X[, y])</code>	Fit the model with X.
<code>fit_transform(X[, y])</code>	Fit the model with X and apply the dimensionality reduction on X.
<code>get_covariance()</code>	Compute data covariance with the generative model.
<code>get_feature_names_out([input_features])</code>	Get output feature names for transformation.
<code>get_metadata_routing()</code>	Get metadata routing of this object.
<code>get_params([deep])</code>	Get parameters for this estimator.
<code>get_precision()</code>	Compute data precision matrix with the generative model.
<code>inverse_transform(X)</code>	Transform data back to its original space.
<code>score(X[, y])</code>	Return the average log-likelihood of all samples.
<code>score_samples(X)</code>	Return the log-likelihood of each sample.
<code>set_output(*[, transform])</code>	Set output container.
<code>set_params(**params)</code>	Set the parameters of this estimator.
<code>transform(X)</code>	Apply dimensionality reduction to X.

- In scikit-learn, PCA is implemented as **probabilistic model**: can estimate the likelihood of data based on the amount of variance it explains.
- **Incremental PCA**: add `partial_fit` method
- **SparsePCA**: introduce sparsity in decomposition

$$(U^*, V^*) = \arg \min_{U, V} \frac{1}{2} \|X - UV\|_{\text{Fro}}^2 + \alpha \|V\|_{1,1}$$

subject to $\|U_k\|_2 \leq 1$ for all $0 \leq k < n_{\text{components}}$

- ... and there is always more (dictionary methods, etc)

Examples: Eigenfaces

- When viewed as vectors of pixel values, face images are extremely high dimensional. Image of 100x100 pixels has 10,000 dimensions.
- However, very few of 100x100 vectors are valid face images
- We want to effectively represent the subspace of face images

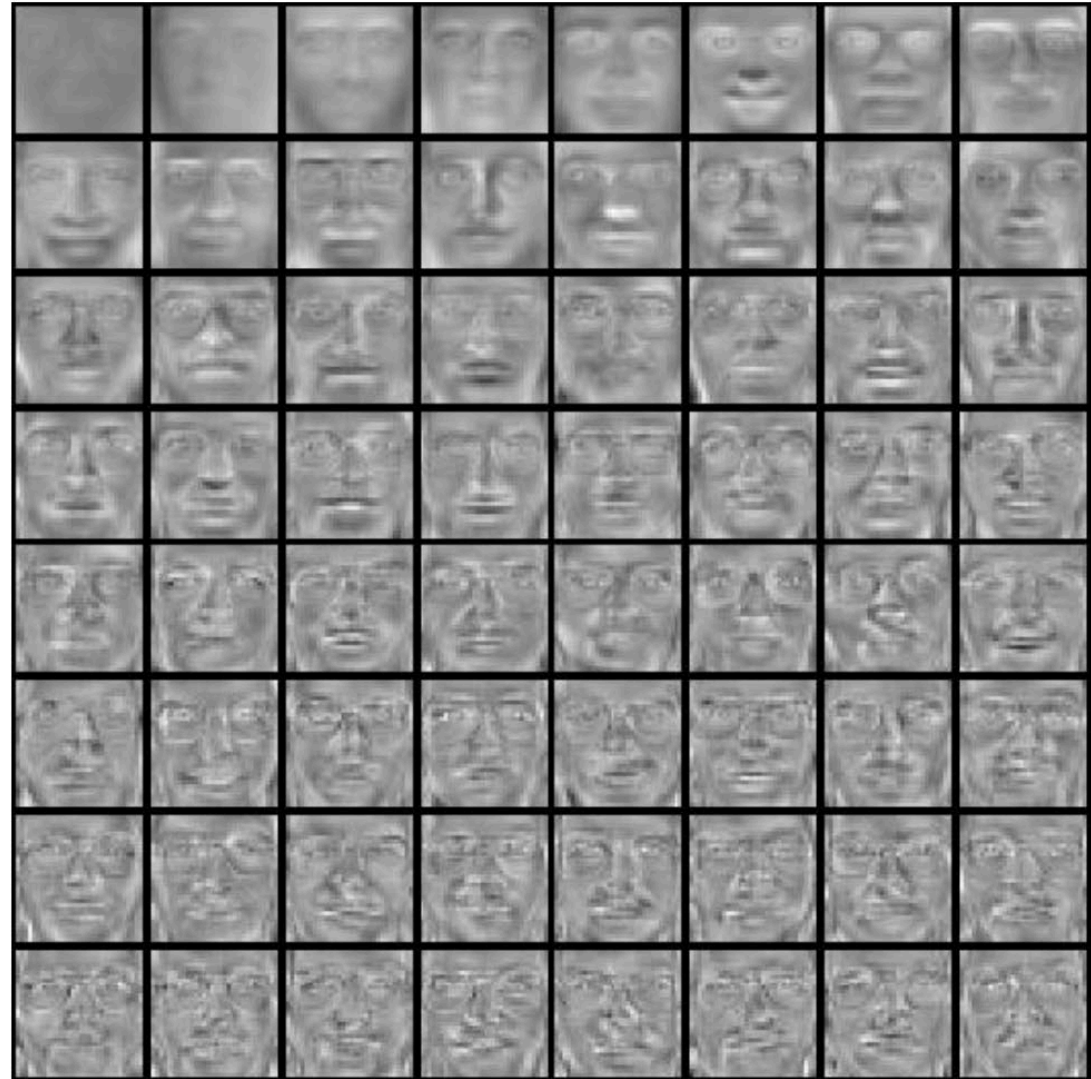


Adapted from Fereshteh Sadeghi
slide by Derek Hoiem

Examples: Eigenfaces

Top eigenvectors: $u_1, \dots, u_k$

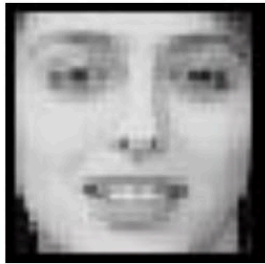
Mean: μ



Adapted from Fereshteh Sadeghi slide by Derek Hoiem

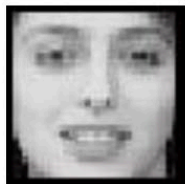
Representation and reconstruction

- Face $\mathbf{x}$ in “face space” coordinates:



$$\mathbf{x} \rightarrow [\mathbf{u}_1^T (\mathbf{x} - \mu), \dots, \mathbf{u}_k^T (\mathbf{x} - \mu)]$$
$$= w_1, \dots, w_k$$

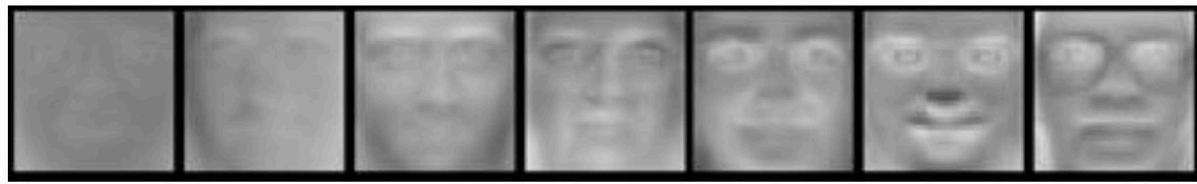
- Reconstruction:



=



+



$\hat{\mathbf{x}}$

=

μ

+

$w_1 u_1 + w_2 u_2 + w_3 u_3 + w_4 u_4 + \dots$

Reconstruction

$P = 4$



$P = 200$

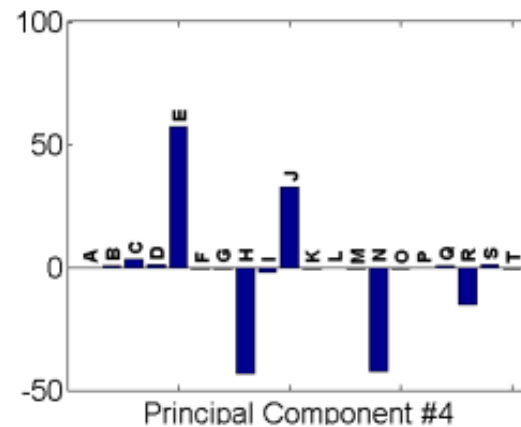
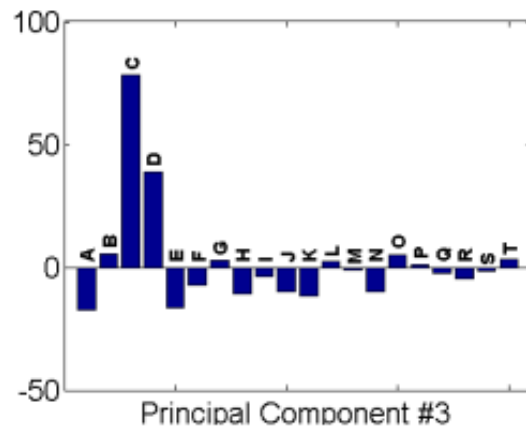
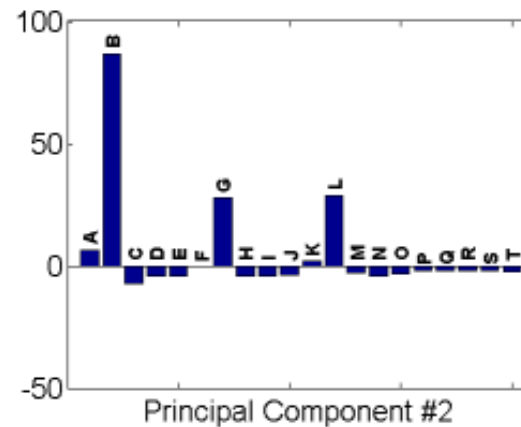
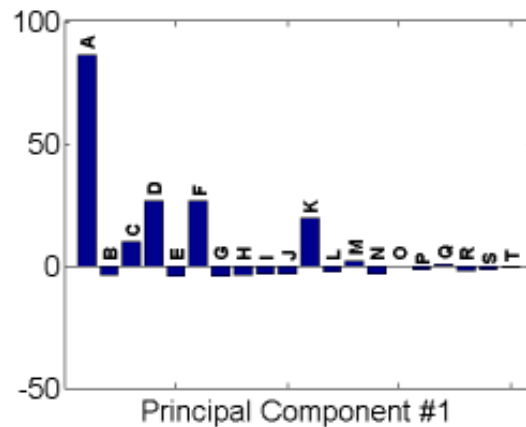


$P = 400$

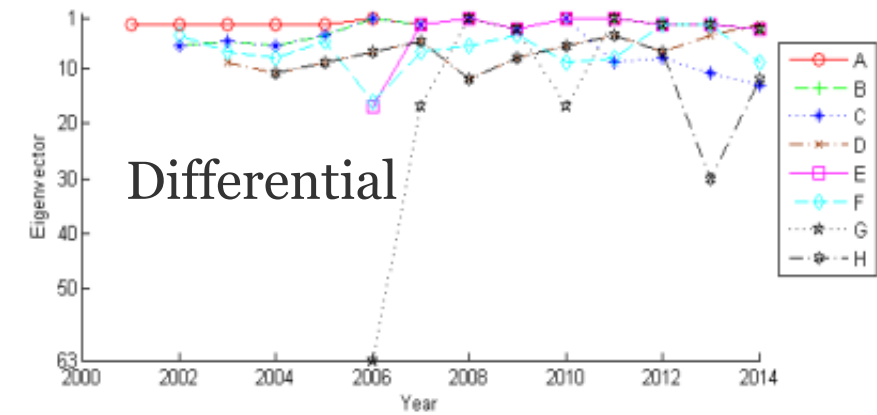
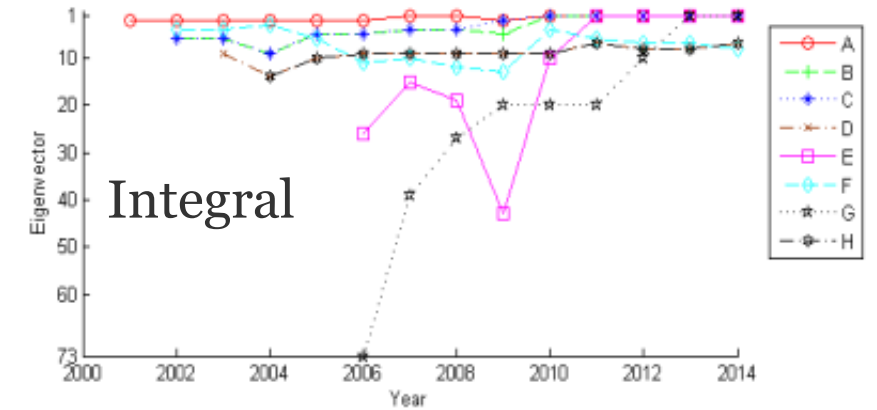


We can represent faces well by 400 components – rather than 10,000!

PCA can be applied to everything



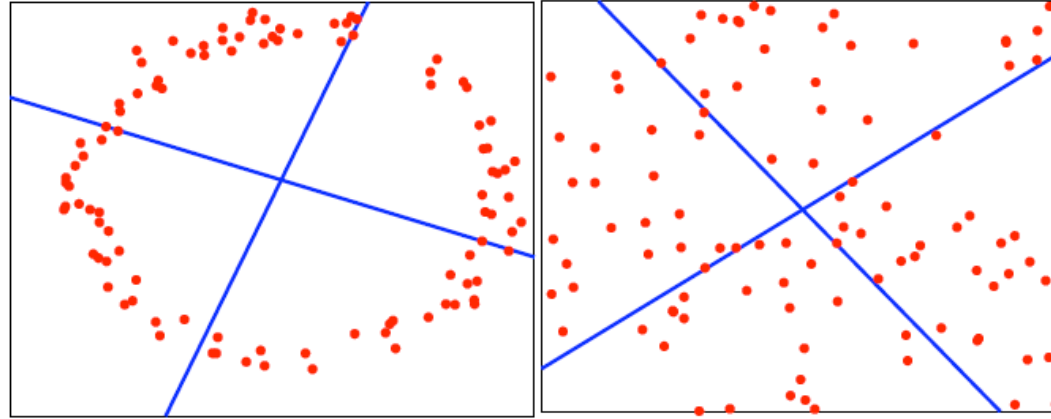
Publication activity



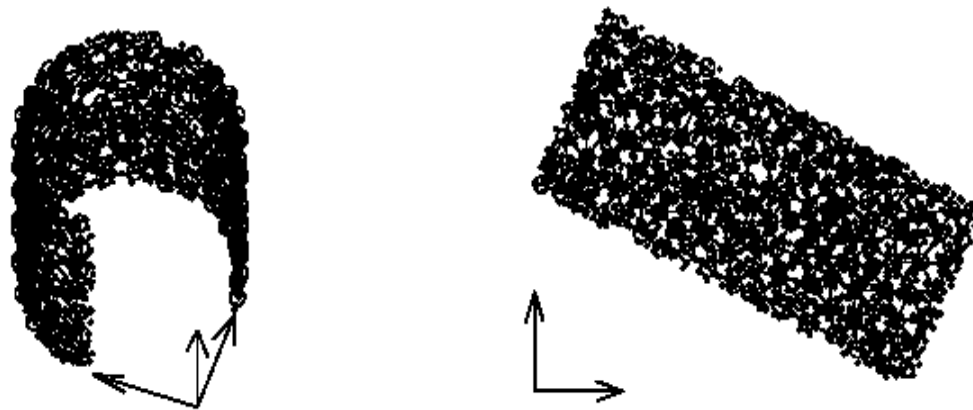
Publication analysis: The dimensionality of space, N , is defined by the total number of scientists. Each publication in this case then defines a single point in this space. For example, for the 28-dimensional space of authors of $\{A, B, C, \dots, Z\}$ the publication authored by A and C will be represented as $\{1, 0, 1, \dots, 0\}$, and by A, B, and Z as $\{1, 1, 0, \dots, 0, 1\}$, where all missing elements are zeroes.

<https://arxiv.org/abs/1502.03439>

Limitations of PCA



PCA will make no difference between these examples



Nonlinear projection of a horseshoe

Non-linear PCA

- Suppose that instead of using the points $\mathbf{x}_i$ as is, we wanted to go to some different feature space $\phi(\mathbf{x}_i) \in \mathbb{R}^N$
- E.g. using polar coordinates instead of cartesian coordinates would help us deal with the circle
- In the higher dimensional space, we can then do PCA
- The result will be non-linear in the original data space!
- Similar idea to support vector machines

Kernel PCA

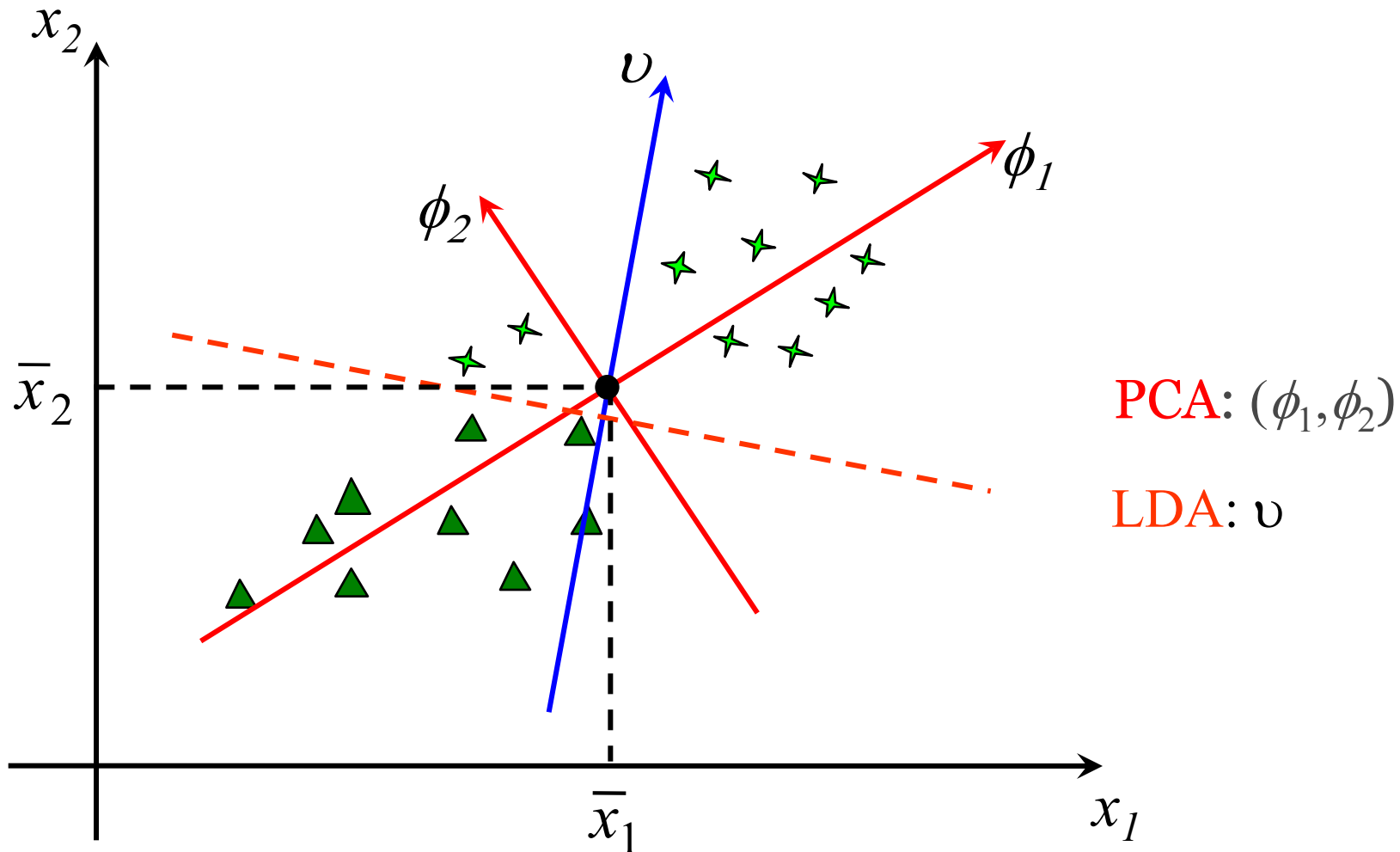
Kernel PCA is an unsupervised manifold learning technique that maps data points to a generally lower-dimensional space. It generalizes the Principal Components Analysis approach to non-linear transformations using the kernel trick (Schölkopf, Smola and Müller, 1996; Schölkopf, Smola and Müller, 1998; Schölkopf, Burges and Smola, 1999). The algorithm implicitly finds the leading eigenvectors and eigenvalues of the covariance of the projection $\phi(x)$ of the data in “feature space”, where $\phi(x)$ is such that the kernel $K_n(x, y) = \phi(x) \cdot \phi(y)$ (i.e. K_n must not have negative eigenvalues). If the data is

(http://research.microsoft.com/users/Cambridge/nicolasl/papers/eigen_dimred.pdf)

Linear Discriminant Analysis

- Linear Discriminant Analysis, or simply LDA, is a feature extraction technique that has been used successfully in many statistical pattern recognition problems.
- LDA is often called Fisher Discriminant Analysis (FDA).
- The primary purpose of LDA is to separate samples of distinct groups by transforming them to a space which maximises their between-class separability while minimising their within-class variability.
- It assumes implicitly that the true covariance matrices of each class are equal because the same within-class scatter matrix is used for all the classes considered.
- If we remove this assumption, we get Quadratic Discriminant Analysis

Geometric Idea of LDA



LDA Steps

1. Compute the d -dimensional mean vectors.
2. Compute the scatter matrices
3. Compute the eigenvectors and corresponding eigenvalues for the scatter matrices.
4. Sort the eigenvalues and choose those with the largest eigenvalues to form a $d \times k$ dimensional matrix
5. Transform the samples onto the new subspace.

LDA Method

- Let the between-class scatter matrix S_b be defined as

$$S_b = \sum_{i=1}^g N_i (\bar{x}_i - \bar{x})(\bar{x}_i - \bar{x})^T$$

- and the within-class scatter matrix S_w be defined as

$$S_w = \sum_{i=1}^g (N_i - 1) S_i = \sum_{i=1}^g \sum_{j=1}^{N_i} (x_{i,j} - \bar{x}_i)(x_{i,j} - \bar{x}_i)^T$$

- where $x_{i,j}$ is an n -dimensional data point j from class p_i , N_i is the number of training examples from class p_i , and g is the total number of classes or groups.

LDA Method

- The sample mean, sample covariance, and grand mean vector are given respectively by:

$$\bar{x}_i = \frac{1}{N_i} \sum_{j=1}^{N_i} x_{i,j}$$

$$S_i = \frac{1}{(N_i - 1)} \sum_{j=1}^{N_i} (x_{i,j} - \bar{x}_i)(x_{i,j} - \bar{x}_i)^T$$

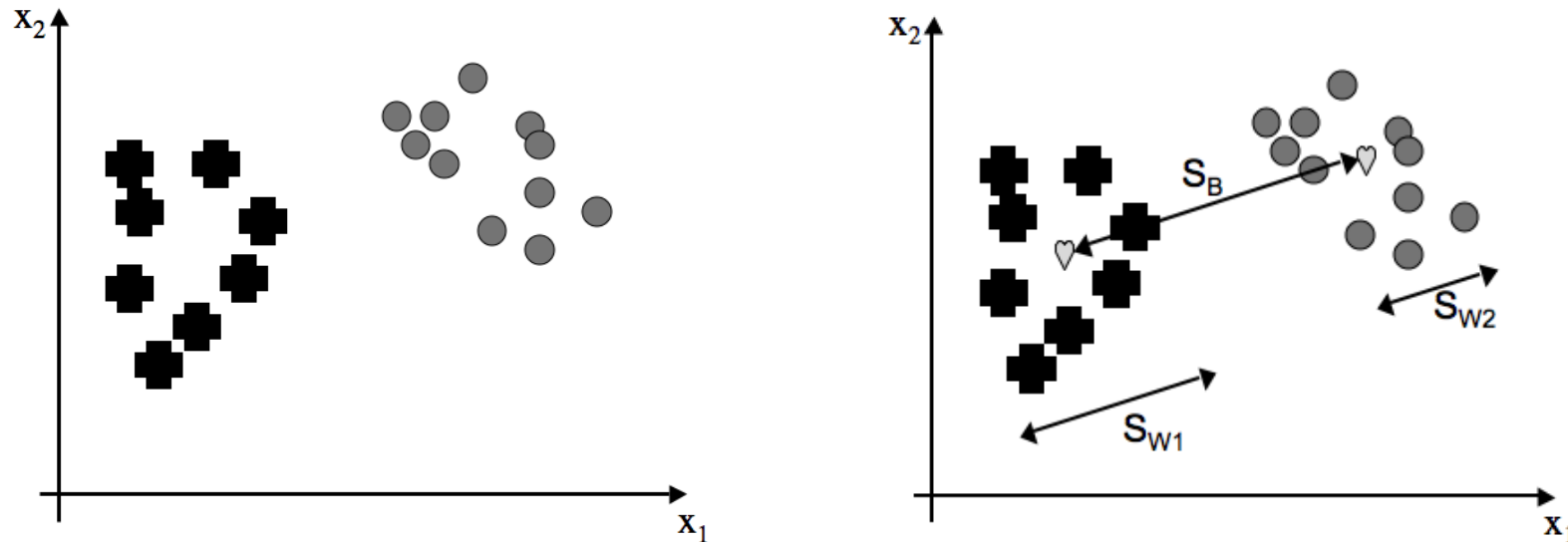
$$\bar{x} = \frac{1}{N} \sum_{i=1}^g N_i \bar{x}_i = \frac{1}{N} \sum_{i=1}^g \sum_{j=1}^{N_i} x_{i,j}$$

$$N = N_1 + N_2 + \dots + N_g$$

LDA Method

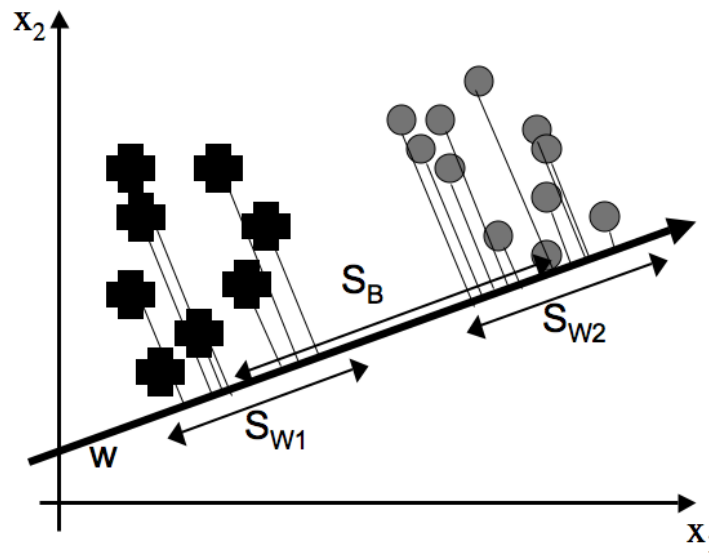
The objective of LDA is to find a projection matrix P_{lda} that maximises the ratio of the determinant of S_b to the determinant of S_w (Fisher's criterion) as:

$$P_{lda} = \arg \max_P \frac{|P^T S_b P|}{|P^T S_w P|}$$

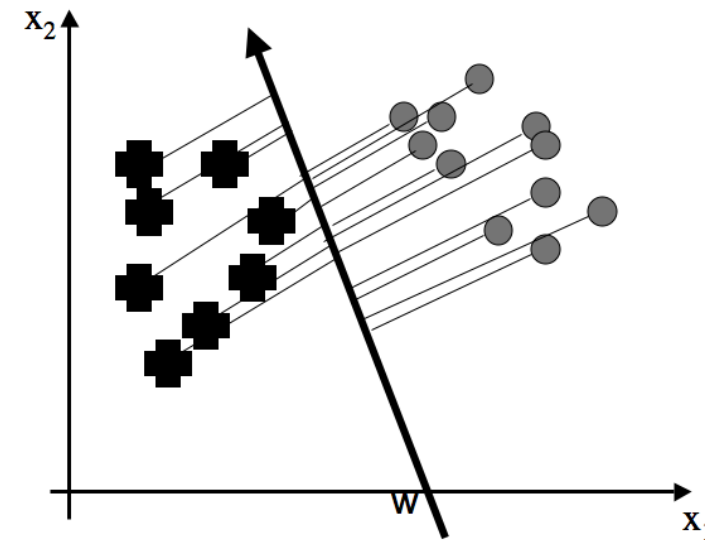


LDA Method

- So Fisher's criterion tries to find the projection that:
 - Maximises the variance of the class means
 - Minimises the variance of the individual classes



Good projection



Bad projection

Classification using LDA

- The LDA is an axis projection.
- Once the projection is found all the data points can be transformed to the new axis system along with the class means and covariances.
- Allocation of a new point to a class can be done using a distance measure such as the Mahalanobis distance.

Given a vector $\mathbf{x}$, a mean vector μ , and a covariance matrix Σ , the Mahalanobis Distance D_M is defined as:

$$D_M(\mathbf{x}, \mu) = \sqrt{(\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu)}$$

where:

- $\mathbf{x}$ is the vector for which the Mahalanobis Distance is being computed.
- μ is the mean vector of the distribution.
- Σ is the covariance matrix of the distribution.
- T denotes a matrix transpose.
- Σ^{-1} is the inverse of the covariance matrix.

LDA vs. PCA

- LDA is supervised ML method, PCA is unsupervised method
- LDA seeks directions that are efficient for *discriminating* data whereas PCA seeks directions that are efficient for *representing* data.
- The directions that are discarded by PCA might be exactly the directions that are necessary for distinguishing between groups.

`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

```
class sklearn.discriminant_analysis.LinearDiscriminantAnalysis(solver='svd', shrinkage=None, priors=None, n_components=None, store_covariance=False, tol=0.0001, covariance_estimator=None)
```

[\[source\]](#)

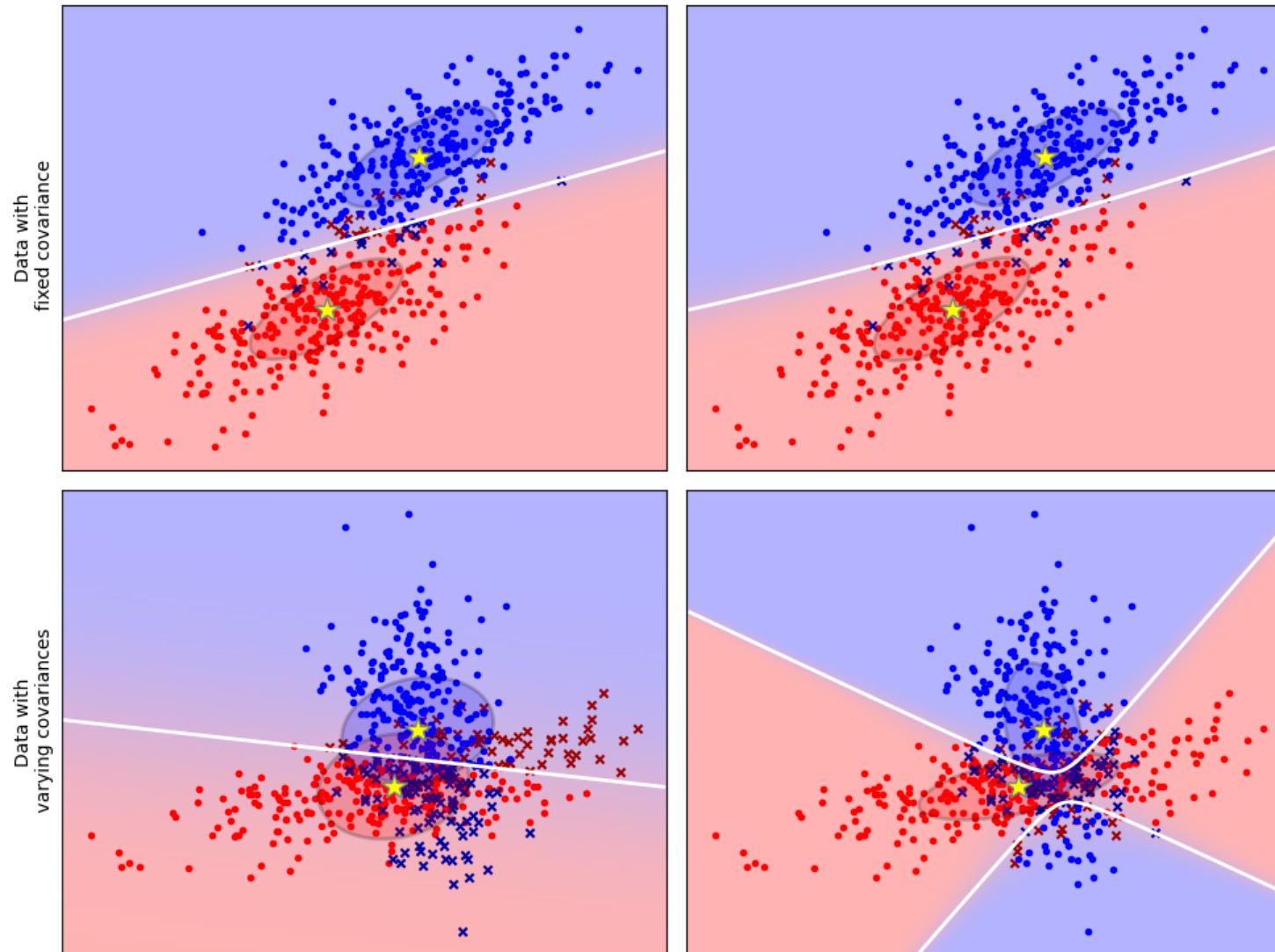
<code>decision_function(X)</code>	Apply decision function to an array of samples.
<code>fit(X, y)</code>	Fit the Linear Discriminant Analysis model.
<code>fit_transform(X[, y])</code>	Fit to data, then transform it.
<code>get_feature_names_out([input_features])</code>	Get output feature names for transformation.
<code>get_metadata_routing()</code>	Get metadata routing of this object.
<code>get_params([deep])</code>	Get parameters for this estimator.
<code>predict(X)</code>	Predict class labels for samples in X.
<code>predict_log_proba(X)</code>	Estimate log probability.
<code>predict_proba(X)</code>	Estimate probability.
<code>score(X, y[, sample_weight])</code>	Return the mean accuracy on the given test data and labels.
<code>set_output(*[, transform])</code>	Set output container.
<code>set_params(**params)</code>	Set the parameters of this estimator.
<code>set_score_request(*[, sample_weight])</code>	Request metadata passed to the <code>score</code> method.
<code>transform(X)</code>	Project data to maximize class separation.

Quadratic Discriminant Analysis

Linear Discriminant Analysis vs Quadratic Discriminant Analysis

Linear Discriminant Analysis

Quadratic Discriminant Analysis



LDA vs. QDA

Bayesian classification framework:

$$P(y = k|x) = \frac{P(x|y = k)P(y = k)}{P(x)} = \frac{P(x|y = k)P(y = k)}{\sum_l P(x|y = l) \cdot P(y = l)}$$

QDA approximation:

$$P(x|y = k) = \frac{1}{(2\pi)^{d/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^t \Sigma_k^{-1}(x - \mu_k)\right)$$

LDA is a special case of QDA, where the Gaussians for each class are assumed to share the same covariance matrix: $\Sigma_k = \Sigma$. If so

$$\log P(y = k|x) = \omega_k^t x + \omega_{k0} + Cst.$$

where $\omega_k = \Sigma^{-1} \mu_k$ and $\omega_{k0} = -\frac{1}{2} \mu_k^t \Sigma^{-1} \mu_k + \log P(y = k)$. These quantities correspond to the `coef_` and `intercept_` attributes, respectively.

Note: Relation with Gaussian Naive Bayes

If in the QDA model one assumes that the covariance matrices are diagonal, then the inputs are assumed to be conditionally independent in each class, and the resulting classifier is equivalent to the Gaussian Naive Bayes classifier `naive_bayes.GaussianNB`.

Spectroscopic Imaging

Scanning probe microscopy:

- Force-distance curve measurements
- Current-voltage measurements
- Piezoresponse force/electrochemical strain spectroscopy

Electron microscopy:

- Electron Energy Loss Spectroscopy

Optical microscopy:

- Hyperspectral imaging
- Time resolved measurements

Mass-spectrometry:

- Secondary ion MS imaging

In many cases, measured signal can be represented or approximated as a linear combination of signals. However, their functional forms are generally unknown

Very important: convolution with resolution function is also mixing

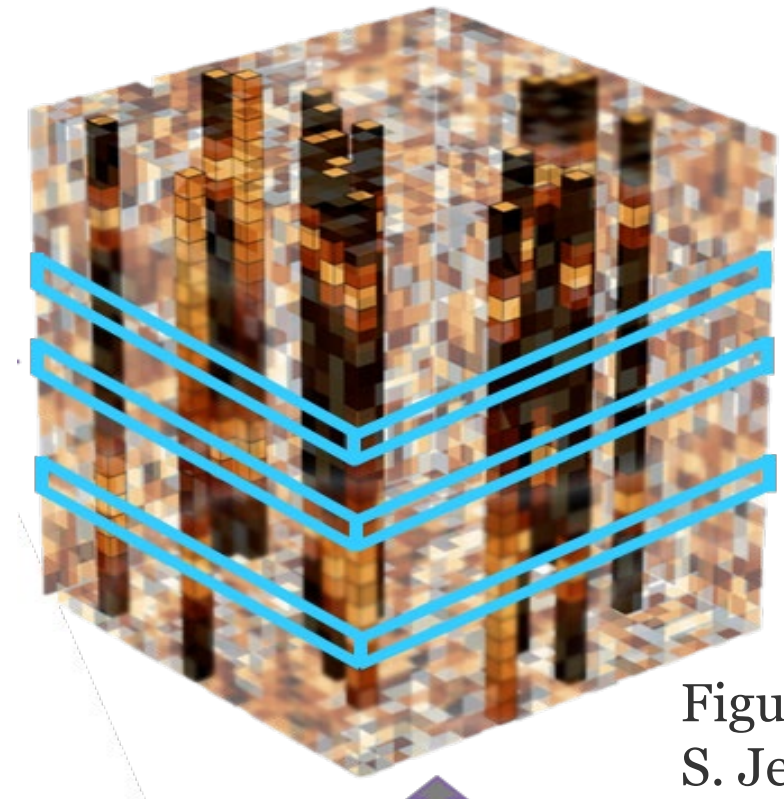
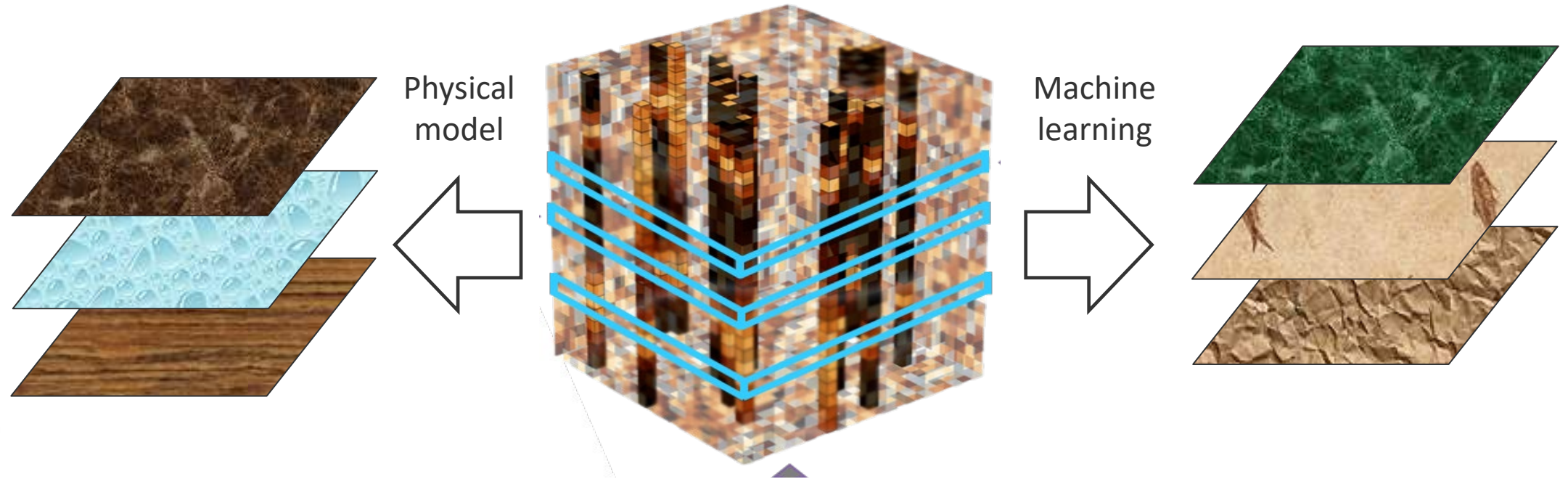


Figure by
S. Jesse

Physics vs. ML based analysis



- If we have physical model, we can extract relevant parameters from data
- Imperfect model: epistemic uncertainty
- Noisy data: aleatoric uncertainty
- Analysis results do not depend on sampling of data in x, y

- If we don't have physical model, we can learn intrinsic structure of data
- **Unsupervised learning:** based on data only
 - But not really (definition of distance)
 - Analysis can depend on sampling of data
- **Supervised learning:** based on prior examples
 - Out of distribution shifts

Physics-informed ML: Combines strengths (and limitations) of both

General linear unmixing

$$S(\mathbf{x}, \mathbf{R}) = \sum_i a_i(\mathbf{x}) w_i(\mathbf{R}) + N$$

We start with:

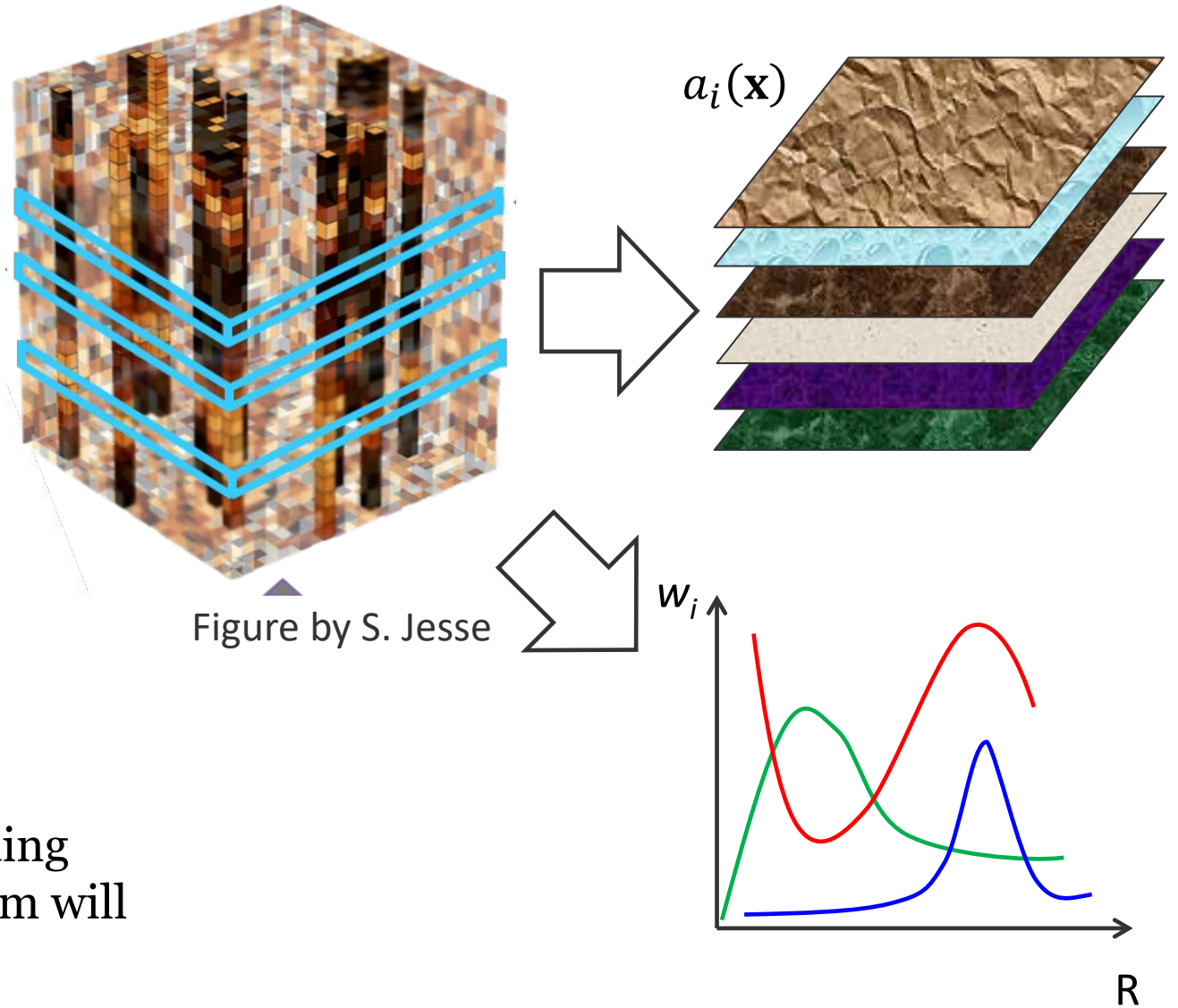
- $\mathbf{x}$ is the spatial variable, $\mathbf{x} = (x, y)$
- $\mathbf{R}$ is the (vector) parameter variable

Overall, for $M \times M$ image and P point in spectra, we have $M^2 P$ data points

We aim to get:

- $a_i(\mathbf{x})$ are loading maps
- $w_i(\mathbf{R})$ are endmembers/eigenvectors
- N is noise

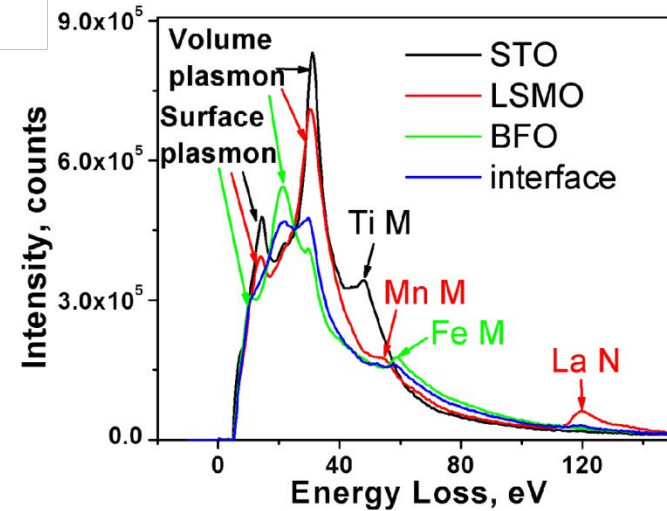
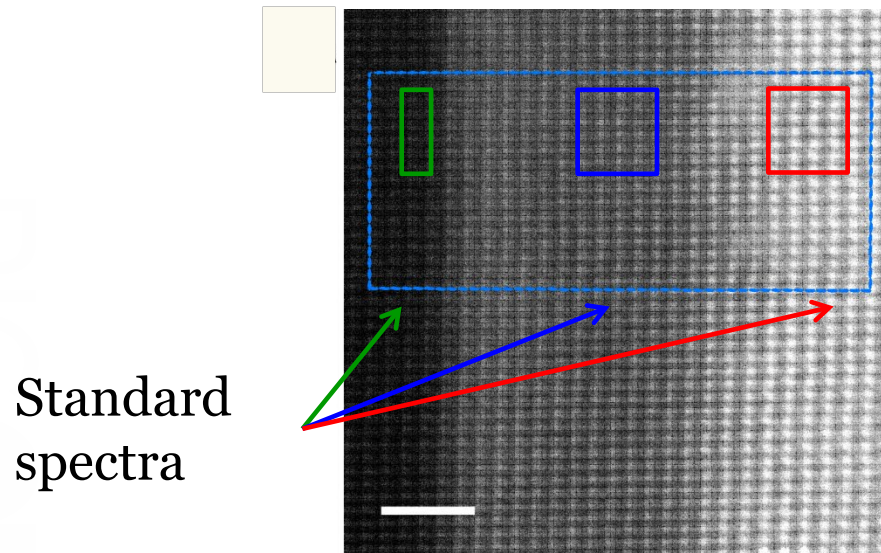
Overall, we can have (maximum) P loading maps of M^2 size. However, not all of them will have useful information



Multiple Linear Regression

Linear mixing $S(\mathbf{x}, \mathbf{R}) = \sum_i a_i(\mathbf{x}) w_i(\mathbf{R}) + N$ but $w_i(\mathbf{R})$ are **known**

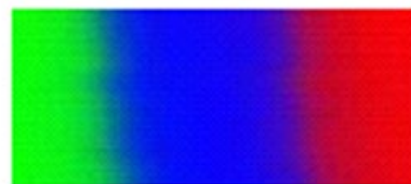
STEM of STO/LSMO/BFO interface Low-loss EELS spectra of three components



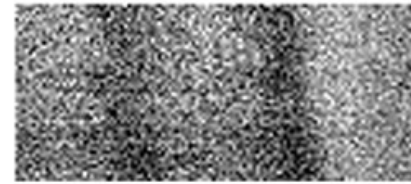
A.Y. BORISEVICH ET AL,
Suppression of Octahedral Tilts and Associated Changes in Electronic Properties at Epitaxial Oxide Heterostructure Interfaces, Phys. Rev. Lett. **105**, 087204 (2010).

“Chemistry”:
 35 to 125 eV

Fit coefficient map



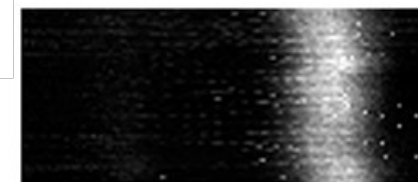
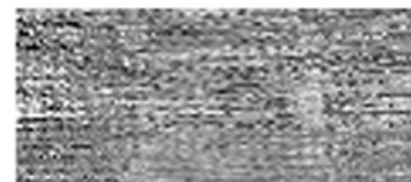
residuals map



χ^2 map



“Plasmons”
 5 to 35 eV



Principal Component Analysis

$$S(\mathbf{x}, \mathbf{R}) = \sum_i a_i(\mathbf{x}) w_i(\mathbf{R})$$

- In PCA, the eigenvectors $w_i(\mathbf{R})$ are orthonormal and are arranged such that corresponding eigenvalues are placed in descending order by variance
- Can be used to separate “real data” from “noise” – but needs cut-off/selection criteria
- PCA eigenvectors generally do not have defined physical meaning
- PCA is a starting point for many other unmixing methods

EELS elemental mapping with unconventional methods I. Theoretical basis: image analysis with multivariate statistics and entropy concepts

Pierre Trebbia *

Laboratoire de Physique des Solides, Bâtiment 510, F-91405 Orsay Cedex, France

and

Noël Bonnet

Unité INSERM 314 et Université de Reims, 21 rue Clément Ader, F-51100 Reims, France

Received 7 June 1990

Electron energy loss filtered images recorded within a transmission analytical electron microscope are now widely used for the mapping of the elemental distribution of a given atomic species in a specimen prepared as a thin film. Such an image processing may produce both valuable results and artifacts if a careful inspection of all the hypotheses needed by the calculation is not carried out. This paper presents some general statistical methods for a contrast information analysis of a noisy image data set. After a brief introduction of different concepts such as contrast, variance, information and entropy, two unconventional approaches for image analysis are explained: the relative entropy computed with respect to a pure random and signal-free image and the factorial analysis of correspondence (a branch of multivariate statistics). In the companion article (part II), these concepts are applied to real experiments and the results compared with those obtained with a conventional method. Although electron energy loss spectroscopy is the only technique considered here, these methods for image analysis can be applied to a wide variety of noisy data sets (spectra, images, ...) recorded from various sources (electrons, photons, ...).

Why historical papers matter:

- 165 1. Often contain elementary introductions
2. Deep insights into principles
3. Surprisingly prescient predictions
4. Comparison with the present: see the big picture

“Those who cannot remember the past are condemned to repeat it.”

George Santayana,
The Life of Reason, 1905



Mapping chemical and bonding information using multivariate analysis of electron energy-loss spectrum images

M. Bosman^{a,*}, M. Watanabe^b, D.T.L. Alexander^c, V.J. Keast^a

^a*Australian Key Centre for Microscopy and Microanalysis, University of Sydney, Sydney, NSW 2006, Australia*

^b*Department of Materials Science and Engineering, Lehigh University, Bethlehem, PA 18015, USA*

^c*Department of Materials Science and Metallurgy, University of Cambridge, Pembroke St. CB2 3QZ, UK*

Received 23 June 2005; received in revised form 26 October 2005; accepted 18 April 2006

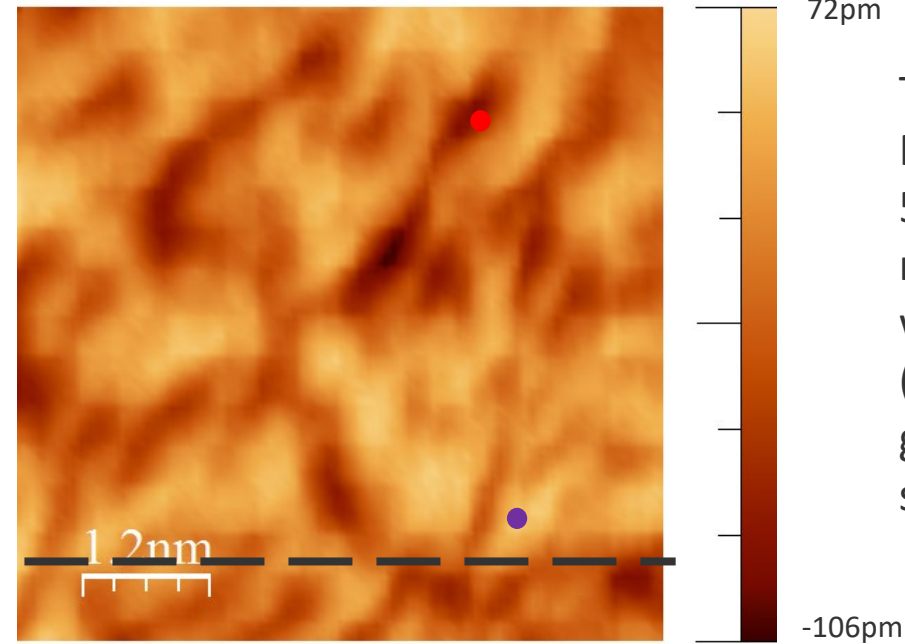
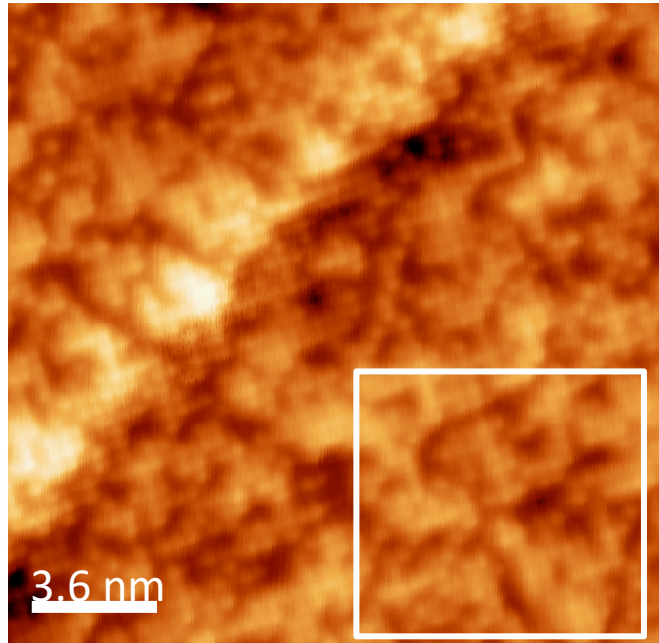
Abstract

Electron energy-loss spectroscopy (EELS) in the transmission electron microscope (TEM) is used to obtain high-resolution information on the composition and the type of chemical bonding of materials. Spectrum imaging, where a full EEL spectrum is acquired and stored at each pixel in the image, gives an exact correlation of spatial and spectral features. However, determining and extracting the important spectral components from the large amount of information contained in a spectrum image (SI) can be difficult. This paper demonstrates that principal component analysis of EEL SIs can be used to extract chemically relevant components. With weighted or two-way scaled principal component analysis, both compositional and bonding information can be extracted. Mapping of the chemical variations in a partially reduced titanium dioxide sample and the orientation-dependent bonding in boron nitride and carbon nanotubes are given as examples.

© 2006 Elsevier B.V. All rights reserved.

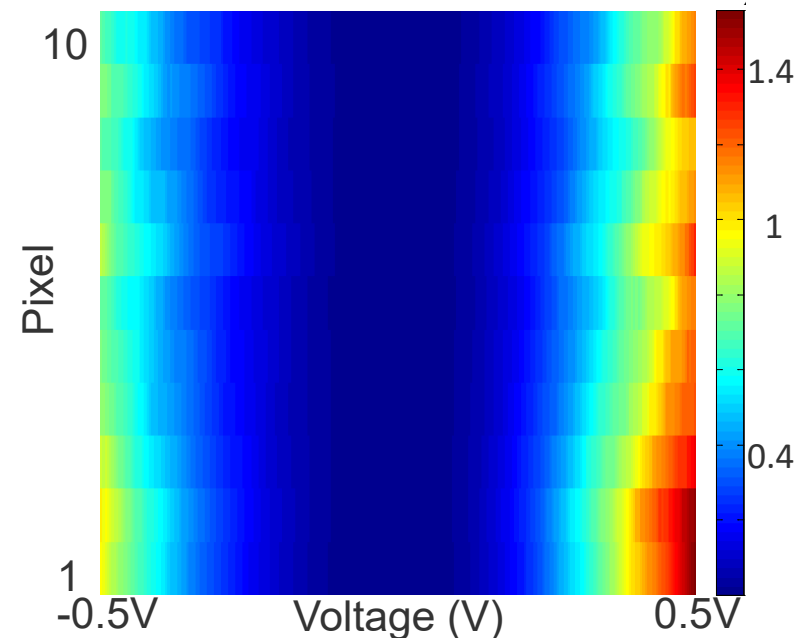
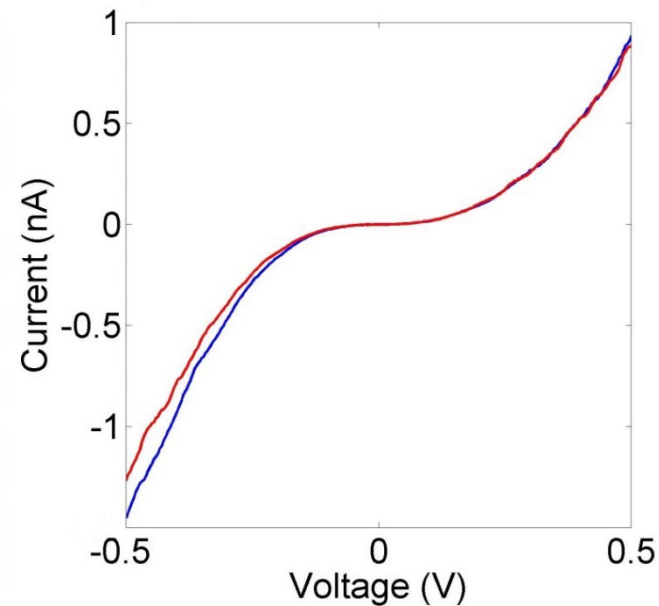
Why did the PCA on EELS started to grow in 2005 – 2010?

Grain boundary by STM



Topographical STM of FeSeTe, $T = 82\text{K}$ $15 \times 15\text{nm}^2$, 50mV, 100pA, white rectangle represents area where CITS was performed;

(b) CITS 80x80 pixel graphical average of the spectrographic data.



M. ZIATDINOV, A. MAKSOV, L. LI, A. SEFAT, P. MAKSYMОВYCH, and S.V. KALININ, *Deep data mining in a real space: Separation of intertwined electronic responses in a lightly-doped BaFe₂As₂*, Nanotechnology 27, 475706 (2016).

Eigenvectors and loadings

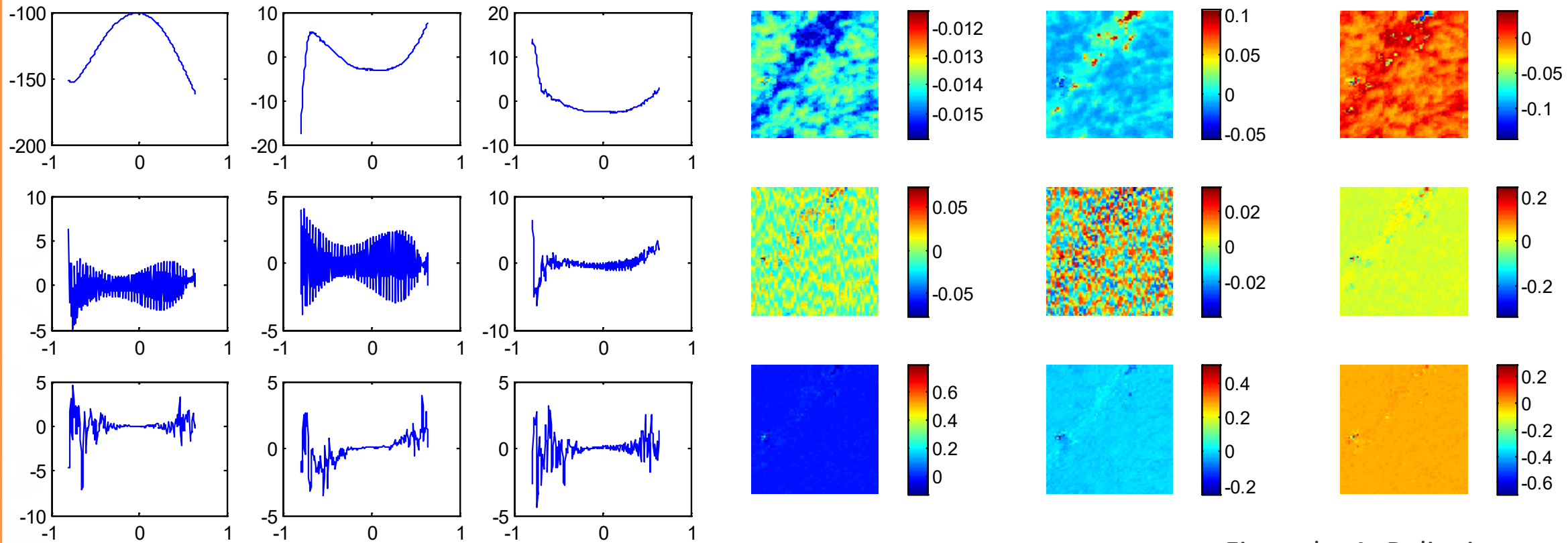


Figure by A. Belianinov